

Converting 2D Images to 3D Models for Laser Subsurface Engraving (SSLE)

In this guide, we'll break down how to turn a single 2D photo into a 3D model suitable for **laser subsurface engraving (SSLE)** inside crystal blocks. We'll cover the entire pipeline from image preprocessing and depth-map creation, through building a 3D mesh, to generating a point cloud that can be used by K9 crystal engraving machines. We'll also explain common 3D file formats (STL, OBJ, XYZ, DXF) and how they differ in terms of quality and usage. Finally, we'll discuss tools like **Blender** and **MeshLab** and how they can help in this workflow. The instructions assume you're using Windows and are new to these techniques, so each step is explained in beginner-friendly detail.

Understanding Key 3D File Formats (STL, OBJ, XYZ, DXF)

Before diving into the process, it's important to understand the output file formats and their differences:

- **STL (stereolithography)** – The most common format for 3D printing and CAD, representing a 3D surface as a mesh of triangles ¹. STL files contain only geometry (no color or texture) and describe the object's surface using many tiny triangles. The more triangles in the mesh, the smoother and higher-quality the surface can appear ². STL is widely supported and used in manufacturing (especially 3D printing) because of its simplicity and universality ³. However, STL has **no concept of units** (it's unit-less) and no additional data like color ⁴.
- **OBJ (Wavefront OBJ)** – A richer 3D model format that can include not only the mesh geometry (and not just triangles, but also other polygons or even NURBS surfaces) but also texture maps, colors, and material properties via a companion `.mtl` file ⁵ ⁶. OBJ can store more detail in terms of visual appearance (for example, a color texture for full-color 3D prints or game assets) ⁷. In terms of geometry quality, **OBJ and STL can represent the same shapes** – an OBJ isn't inherently higher resolution than an STL unless it uses special features like curved surfaces. In practice, most 3D tools (and laser software) will triangulate OBJ data too. As one expert explains, *"there isn't much difference, in terms of the final quality... between an OBJ and an STL. Both files store data in triangles... the same model, be it in STL format or OBJ, will result in the same geometry."* ⁸ The main advantages of OBJ are the support for color/texture and the ability to store quads or curves (which is more useful during modeling than for the final laser engraving).
- **XYZ (Point Cloud)** – A simple text-based format listing point coordinates (X, Y, Z) in 3D space. This is **not a mesh**, but a **point cloud** format: essentially a list of points without connectivity. Many 3D scanners output .xyz files as they are easy to read and "future-proof" (just columns of numbers) ⁹. The downside is XYZ doesn't include any additional info – no color, no normals, etc., just the positions of points ¹⁰. For our purposes, an XYZ file would contain the cloud of laser engraving points inside the crystal. It's a convenient way to deliver those points to the engraving software if it accepts point clouds.

- **DXF (Drawing Exchange Format)** – A CAD format from Autodesk, typically used for 2D drawings (laser cutting, CNC, etc.) ¹¹. DXF is vector-based and can contain lines, curves, and points with precise coordinates ¹². **In SSLE workflows, DXF is sometimes used to convey 3D point data** by using DXF “POINT” entities for each 3D point. In general, DXF isn’t a mesh format and doesn’t natively encode a full 3D surface (it’s mostly 2D or 2.5D), but it’s widely supported in industry software ¹³. Some laser engraving vendors accept a DXF file with points as input for their machines (since many laser software are built around CAD). If using DXF for point clouds, the quality of the result will depend on the points you include (spacing, number of points), not on DXF itself – DXF is just a container for the coordinates.

Which format offers the highest 3D quality? It’s not the file extension alone that determines quality – it’s how finely you represent the shape:

- For **meshes (STL/OBJ)**: Quality comes from having enough resolution (triangles) to capture details. An OBJ can technically use smooth curved surfaces or more complex polygons, but ultimately engraving machines or slicers will convert everything to points or triangles anyway ⁸. So a high-detail STL and a high-detail OBJ can contain the same quality level. Neither inherently gives “better 3D quality” than the other when it comes to geometry – the mesh density and accuracy do that. (One note: OBJ can store the model’s original quad topology or NURBS which is useful for editing or scaling later without loss ¹⁴, but for the printed/engraved output, this gets tessellated into triangles at some stage.)
- For **point clouds (XYZ/DXF)**: Quality depends on the number of points and their distribution. A very dense point cloud can represent fine detail, but a laser machine has limits on how close points can be (too close and the laser points will merge or damage the glass ¹⁵). Conversely, too few points and the engraved image will look faint or jagged ¹⁶. So there is an optimal “quality” where points are spaced just right to give a clear image. This spacing is typically on the order of tenths of a millimeter between points in glass to avoid overlaps ¹⁷ ¹⁵. Using techniques like Poisson-disk sampling ensures an even spacing so you don’t get clusters or gaps in the point cloud (we’ll cover this later) ¹⁸.

In summary: **STL is great for universally sharing a 3D shape**, OBJ is useful if you need color or want to preserve editable geometry, and point cloud formats (XYZ, DXF) are what the laser engraving machine ultimately needs (a cloud of points inside the crystal). For highest quality engraving, you will focus on generating a very clean mesh and then a well-distributed point cloud – the file format you use to deliver that (OBJ vs STL vs XYZ vs DXF) won’t change the appearance as long as the content is the same. Many SSLE vendors accept an OBJ or STL and do the point conversion themselves, or accept a DXF/XYZ of points directly ¹⁹.

Setting Up Your Development Environment (Windows)

We’ll be using Python for the core of this pipeline. Here’s how to set up everything you need:

1. **Install Python and a Code Editor:** You mentioned you have Python installed. Ensure you have a **Python 3.10+** environment (many of the libraries, especially machine learning ones like PyTorch, work best on 3.10 or newer). You can use an editor like VSCode or PyCharm to write your scripts.

2. **Create a virtual environment (optional but recommended):** This keeps the project's libraries separate. In Windows, you can open Command Prompt (or PowerShell) and do:

```
python -m venv ssle-env  
ssle-env\Scripts\activate
```

This will activate a virtual environment named "ssle-env". (In PowerShell, the activation command might be `ssle-env\Scripts\Activate.ps1`.)

1. **Install required Python libraries:** Use pip to install the libraries needed for each step:
2. **Image processing and general:** `opencv-python` (for image loading and processing), `numpy` (for array manipulations).
3. **Upscaling (Super-Resolution):** For Real-ESRGAN, there's a convenient package `py-real-esrgan`. Install it along with its deep learning backend. We'll also need PyTorch for any AI models.
4. **AI depth estimation:** We will use **PyTorch** and possibly models like MiDaS or ZoeDepth. Install `torch` and `torchvision` (follow the [official instructions](#) appropriate for your system, especially if you want CUDA GPU support).
5. **Background removal:** `rembg` is a great tool (it uses neural networks to remove background). It may require an additional engine like `onnxruntime`. For CPU usage, you can install it directly with one command.
6. **3D libraries:** `open3d` (for 3D geometry like meshes and point clouds), `trimesh` (another 3D geometry library), and `ezdxf` (to write DXF files if needed). Additionally, `pymeshlab` or `meshio` can be handy for some conversions, but likely not required if using `open3d`.
7. **Optional face landmark:** `mediapipe` (if you want to use it for face detection/landmarks).
8. **Optional advanced:** If you ever use 3D Morphable Models for faces, libraries like `face-alignment` or specific 3DMM packages would be needed, but you can skip these for now.

A combined pip install command (after activating your env) could be:

```
pip install opencv-python numpy torch torchvision open3d trimesh ezdxf rembg  
mediapipe py-real-esrgan
```

(This installs a lot. You might need to install PyTorch separately if pip has trouble finding a pre-built wheel; the PyTorch site can give you a pip command with the right CUDA version.)

Note: Installing `rembg` via pip will by default include the necessary model for background removal. For example, `pip install rembg[cpu]` installs it with CPU support ²⁰. If you have a GPU and want it faster, you'd install `rembg[gpu]` and ensure you have the proper CUDA libraries ²¹. For now, CPU mode is fine.

1. **Install Blender and MeshLab:** Since you already did this, just ensure they run properly. Blender (version 4.x in your case) and MeshLab don't need Python, but we might use them manually outside

of the script for visualization. No special setup beyond the normal installation is needed for our purposes.

With the environment ready, let's go through each step of the process in detail.

Step 1: Preprocessing Your Image

Preprocessing improves the input photo so that the depth estimation and 3D conversion give the best results. We will do the following sub-steps: **denoise/upscale the image**, **remove the background**, and (for portraits) possibly **normalize the face position/size**.

1A. Denoising and Upscaling the Image

A clean, high-resolution image yields a more detailed and smoother depth map. Upscaling can add detail (or at least preserve small details as we process the image). Denoising can reduce unwanted artifacts that might confuse the depth predictor or appear as bumps on the final model.

Upscaling (Super Resolution): One popular tool for this is **Real-ESRGAN**, an open-source AI model that enhances images. Real-ESRGAN is trained to not only increase resolution but also **remove noise and artifacts while enhancing details** ²². This is great for our use-case if the photo is low-res or a bit grainy. There are also commercial tools like Topaz Gigapixel AI, but Real-ESRGAN can be used free via Python or even a pre-built app.

Using Real-ESRGAN in Python is straightforward. You can use the `py-real-esrgan` package which provides a simple interface. For example:

```
from PIL import Image
from py_real_esrgan.model import RealESRGAN

# Load the Real-ESRGAN x4 model (4x upscaling)
model = RealESRGAN(device="cuda" or "cpu", scale=4) # use "cuda" if GPU
available
model.load_weights('weights/RealESRGAN_x4.pth', download=True)

img = Image.open("photo.jpg").convert("RGB")
sr_img = model.predict(img)
sr_img.save("photo_upscaled.png")
```

This would take **photo.jpg** and produce **photo_upscaled.png** with 4× the resolution (and hopefully cleaner detail) ²³ ²⁴. If you don't have a GPU, you can set device to CPU (it will be slower). Alternatively, Real-ESRGAN has a command-line tool or even a Windows GUI in some forks, which might be easier if you prefer not to code this part. Upscaling by 2× or 4× is common. Keep in mind, beyond a point there are diminishing returns – 4× upscale can capture more detail for depth conversion as one experiment found ²⁵, but it also **slows down processing** proportionally (4× width/height means 16× more pixels to process in the depth step).

Denoising: If your image has a lot of noise (e.g., from a phone in low light), you might apply a mild denoise filter. However, **be cautious not to over-smooth the image**. A community experiment noted that aggressive denoising blurred important edges and actually *degraded* the final 3D result ²⁶. Only use denoising if needed, and prefer methods that preserve edges. Some upscalers like Real-ESRGAN inherently do some denoising as part of their enhancement ²², so often you get a cleaner image already.

Practical tip: You can use OpenCV's `cv2.bilateralFilter` or non-local means denoising for mild noise removal, which try to keep edges sharp. But if the source photo looks fine, you can skip explicit denoising.

1B. Removing the Background (Segmentation)

For a laser-engraved crystal, you typically want the **subject** (person, pet, object) to stand out, without the background points cluttering the image. So we create a **mask** of the subject and remove the background.

Rembg (Remove Background): This is an AI tool that automatically finds the foreground. It's easy to use – after installing, you can use it via command line or Python. For instance, in Python:

```
import cv2
from rembg import remove

img = cv2.imread("photo_upscaled.png")
output = remove(img) # returns image with background removed (transparent)
cv2.imwrite("photo_no_bg.png", output)
```

This will produce an image where the background is transparent (or white/black depending on how you handle alpha). `rembg` supports different models, but by default it uses a pre-trained one (U2Net) that works for general cases. As the documentation states, “Rembg is a tool to remove images background” using deep learning ²⁷. Under the hood, it finds the main object and cuts it out.

Soft edges (feathering): After background removal, the subject's edges might be too sharp an outline. In 3D, a harsh edge can create an unnatural wall or sudden drop in the depth. It's good to **feather the mask** a bit – i.e., blur the alpha channel slightly so the transition from subject to removed background is smooth. You can do this by taking the alpha channel of `photo_no_bg.png` (if rembg outputs an RGBA image) and applying a Gaussian blur, then recombining it. Even a 5-10 pixel blur can help create a smoother boundary in the depth map and final model.

If you prefer not to code this, you could manually touch up the mask in an image editor (Photoshop/GIMP), painting a soft edge around the subject. But programmatically: OpenCV can blur the alpha channel or you can use PIL to apply a filter.

Why remove background? Because most photos have busy backgrounds that would translate into unnecessary or confusing depth data. By removing it, you essentially set the background to “far away” or blank. Later we'll likely assign a constant far depth to everything outside the subject so that the background in the final crystal is either flat or not prominent. This makes the main subject “pop” as a 3D engraving, and also avoids wasting laser points on background noise.

1C. Face Landmarks and Alignment (Portraits, Optional)

If you are processing **portraits (people's faces)** regularly, it's beneficial to normalize them so that, for example, the eyes are always at a certain level and the face is a certain size in your frame. This ensures consistency in depth scaling (so one face doesn't come out unnaturally larger or smaller than another in the crystal) and also helps the depth algorithm focus on the face details.

Tools like **MediaPipe** (Google's solution) or `face-alignment` (a Python library using PyTorch) can detect key facial landmarks (eyes, nose, mouth corners, etc.). With those landmarks, you could, for example, rotate the image slightly if the face is tilted, scale the image so the distance between the eyes is a given constant (say 35 mm in the real world scale of the crystal), and center the face. This step is more advanced and not strictly required – you can get good results without it. But professionals often do some manual or automated alignment: e.g., ensure the head is not cropped weirdly, and maybe adjust the perspective if the face is at an angle (though that's harder automatically).

If you want to try it: MediaPipe has a Python solution that returns landmarks. You could then warp or transform the image. However, if you're a beginner, you might skip this for now. Instead, you can manually ensure the subject's face is roughly upright and well-cropped in the photo before feeding it in.

Summary for Step 1: By the end of preprocessing, you should have an image (let's call it `preprocessed.png`) that is high-resolution, with only the subject visible (background transparent or a flat color), nicely centered and scaled. This image is ready to be turned into a depth map.

Step 2: Depth Estimation (Creating a Depth Map from the Image)

What is a depth map? It's a grayscale image where each pixel's intensity represents distance from the camera: typically, white = close and black = far (or vice versa). We need a depth map because the laser engraving's 3D effect comes from understanding what parts of the photo are "foreground" versus "background" in 3D space. A good depth map will, for example, make a person's nose and forehead closer (lighter), and the ears and background farther (darker), creating a 3D relief when interpreted.

We will use an **AI model for monocular depth estimation** – meaning it takes one image and predicts a depth map. There are several state-of-the-art models available:

- **MiDaS (DPT)** by Intel ISL – A robust model trained on many datasets ²⁸. It produces **relative depth**, which is fine for us (we only need relative distances). MiDaS has multiple variants; for example "DPT-Large" is a high-accuracy model (with a Vision Transformer backend) and "MiDaS small" is a lightweight model ²⁹. The larger the model, the better the detail, but the slower it runs. MiDaS v3 (DPT models) gives excellent results on a wide range of images.
- **LeReS** (Left/Right Depth) – another model, known for good performance on general scenes.
- **ZoeDepth** – a newer model that can predict metric depth (absolute scale) and often preserves edges well. It's a bit more complex to set up but has great quality.
- Others: **DepthLab (Google), PIXMASH, etc.** – but MiDaS and ZoeDepth are among the top easy-to-use ones as of now.

For our pipeline, let's use **MiDaS** due to its ease of use via PyTorch Hub. Once PyTorch is installed, you can load MiDaS in just a few lines as given by the PyTorch Hub example:

```

import torch
import cv2
# Load the MiDaS model (choose a model type)
model_type = "DPT_Large" # highest accuracy, slower
midas = torch.hub.load("intel-isl/MiDaS", model_type)
midas.to(device) # device = torch.device('cuda' or 'cpu')
midas.eval()

# Load transforms to prepare the image
midas_transforms = torch.hub.load("intel-isl/MiDaS", "transforms")
transform = midas_transforms.dpt_transform if "DPT" in model_type else
midas_transforms.small_transform

# Read the preprocessed image
img = cv2.imread("preprocessed.png")
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
input_batch = transform(img_rgb).to(device)

# Perform inference
with torch.no_grad():
    prediction = midas(input_batch)
    prediction = torch.nn.functional.interpolate(
        prediction.unsqueeze(1),
        size=img_rgb.shape[:2],
        mode="bicubic",
        align_corners=False
    ).squeeze()

depth_map = prediction.cpu().numpy()
cv2.imwrite("depth_raw.png", (depth_map * 255).astype('uint8'))

```

This code loads a MiDaS model and runs it on the image ³⁰ ³¹ . A few notes on this:

- We choose `DPT_Large` for best quality. If this is too slow on your machine (or you run out of VRAM on GPU), try `DPT_Hybrid` or even `MiDaS_small` . The PyTorch Hub description confirms these model choices correspond to a trade-off between speed and accuracy ²⁹ .
- We apply a transform that resizes and normalizes the image appropriately for the model ³² .
- The output `prediction` is a 2D tensor of relative inverse depth. We upscale it back to the original image resolution (with interpolation) and get a NumPy array.
- We then save it as an 8-bit image (`depth_raw.png`) for inspection. Initially, this depth might not be nicely contrast-stretched, but we'll post-process it.

Post-processing the depth map: The raw depth map from MiDaS might have a large dynamic range or be inverted (MiDaS typically gives higher values for closer objects, but as “inverse depth”). We should normalize it and integrate our background mask:

- Use the mask from Step 1B: wherever the background was removed, we want to set depth to “far”. That likely means making those pixels *dark* in the depth map (if we use white = close). We can take the alpha or mask and apply it: for example, if the mask is 0 for background, we can force `depth_map[pixels_where_mask_is_background] = max_depth_value`.
- Normalize the depth values to span 0 to 1 (or 0 to 255 in uint8). You can do:

```
depth = depth_map - depth_map.min()
depth = depth / depth.max()
cv2.imwrite("depth_norm.png", (depth*255).astype('uint8'))
```

Now `depth_norm.png` is a normalized depth map.

- **Smooth the depth map:** Often the predicted depth has some noise or rough edges. Applying an *edge-preserving smoothing* is useful so that flat areas stay flat but without losing edges. A **bilateral filter** on the depth map can achieve this – it smooths small variations while keeping sharp edges (since we want to preserve edges around features like the nose or chin). OpenCV has `cv2.bilateralFilter()` which you can use on the depth image (you might want to scale depth to 0–255 and use that as a pseudo image for filtering, then scale back).
- **Clamp background depth:** If the background in the photo was very distant or not well-defined, MiDaS might output it as just “very far”. You can make a decision to clamp it at some depth. For example, in a portrait, you might decide that anything beyond the subject is just a flat background at the farthest distance. This could involve finding the depth value corresponding to the background region and setting it uniformly. This ensures the background doesn’t have random depth variation that could cause “curled” edges or bumps. Essentially, make the background a flat plane behind the subject.

After these adjustments, you should have a nice `depth.png` that shows, in grayscale, the depth. White (or maybe light gray) will be the nearest points (like tip of the nose, or foreground object) and black/dark will be the farthest (the background, or farthest part of subject). You can open `depth.png` to visually check if it makes sense. Sometimes you might want to invert it (depending on how you plan to use it to displace the mesh). If you want the bright parts to protrude out of the crystal (which is typical: closer = more outward in the relief), then make sure your depth map is brighter for closer. If it’s reversed, just invert the grayscale.

Checkpoint: At this stage, we have a depth map image. All the heavy AI part is done. Next, we’ll turn this depth into an actual 3D shape (mesh).

Step 3: Generating a 3D Relief Mesh from the Depth Map

The depth map is essentially a height field: we can treat the grayscale values like heights and displace a surface accordingly. The plan is to create a 3D mesh (an array of vertices connected as triangles) representing a **bas-relief** of the image. This mesh can then be exported as STL/OBJ.

Here's how to do it:

3A. Create a Plane Mesh with the Correct Aspect Ratio and Size

We need an initial flat plane that will be deformed by the depth values. The plane should have the same aspect ratio as the image and a sufficient resolution (dense grid) to capture the detail from the depth map.

For example, if the final crystal engraving will be an **80 mm x 60 mm** front face, that's our target size for the relief in width and height. We also decide how deep the relief will be – say about **20 mm** at most (faces usually look good with ~20 mm depth in a 60 mm thick block).

- **Plane size:** 80 × 60 mm. In your code or modeling tool, you should size the plane to these dimensions. Coordinate-wise, it could span $x = [0, 80]$, $y = [0, 60]$ (in mm units). Keep track that STL has no unit, but we will *treat* numbers as mm consistently.
- **Mesh resolution:** The plane needs to be subdivided into many small rectangles (which become triangles) so that when we displace by the depth map, we get smooth detail. You could aim for at least one vertex per pixel of the depth map, but that can be heavy. A common approach is to match the depth map resolution. If your depth map (or original image) is, say, 1024×768 pixels, you might create a grid roughly around that resolution (maybe slightly less if needed for performance). In code, you can sample the depth map to a slightly lower resolution if extremely high. The key: **more triangles = smoother mesh**, but also larger file and more to process.

If using **Open3D** or **Trimesh** in Python, you might generate a grid of points and faces manually or use built-in functions. Trimesh, for instance, has utilities for making basic shapes. You can also generate vertices as:

```
import numpy as np
W, H = 80.0, 60.0 # mm
resX, resY = 400, 300 # number of divisions along width and height (for
example)
xs = np.linspace(0, W, resX)
ys = np.linspace(0, H, resY)
# create grid of coordinates
grid_x, grid_y = np.meshgrid(xs, ys)
# these are vertex coordinates in 2D so far
```

Then you'll turn these into vertices and faces (two triangles per grid square). However, there's a simpler route: **use Blender** or another modeling tool to generate a subdivided plane. Since you have Blender, you could do: - Create a plane of size 80×60 mm. - Subdivide it (Blender's edit mode: right-click subdivide, or add a Subdivision Surface and apply, etc.) until it has a dense mesh (you might aim for thousands of vertices). -

Alternatively, use a **Displace modifier** in Blender with the depth map – Blender can directly create the relief from an image (more on this in the Blender section).

But let's continue with the concept assuming a programmatic approach.

3B. Displace the Plane Using the Depth Map

Now, assign each vertex of the plane a **Z position** according to the depth map. If white = near, you'll have white areas get a high Z (out towards viewer), and black areas get low Z (farther back). Decide on the maximum relief depth, e.g. **Z_max = 20 mm**.

Using the normalized depth (0 to 1) from earlier:

```
# Assuming depth is a 2D numpy array matching grid_x, grid_y resolution:
Z_max = 20.0 # 20 mm max
z_values = depth * Z_max # depth was 0 to 1
```

Then, for each grid point (i,j), the vertex becomes `(x_i, y_j, z_values[j,i])`. If your `depth` array size matches `grid_x`, they will align. If not, you might need to sample the depth map to the grid's resolution (using interpolation).

After this, you have a set of 3D points. You then need to form triangles (faces) between them. Typically, each cell of the grid forms two triangles. (This is easier in code with loops or using tools; but to keep it high-level, just know we have made a bumpy surface).

One important tweak: **Pin the borders**. The edges of the photo (where the mask might fade out to background) could have some depth value. If the edges of the mesh are not all at the same baseline, you'll get a "frame" or warped border. It often looks better to have the border of the relief flat or slightly recessed. We can achieve this by setting the depth value to 0 (or a minimum) at the outer border vertices. Some techniques: - Make a thin margin around the image where depth is forced to 0. - Taper the edges by multiplying the depth by a feather mask that goes to zero at the boundaries (like a vignette). - Or simply after displacing, manually set all vertices on the outer boundary (min or max x/y) to $z = 0$.

This prevents the edges of the plane from curling up or down. So the relief transitions to a flat surface at the boundary.

3C. Smooth the Mesh to Remove Pixelation

Even with a good depth map, the surface may be a bit jagged due to high-frequency noise (or just the pixel stepping). We apply a **smoothing filter on the mesh geometry**. A recommended method is **Taubin smoothing**, which is a curvature flow that smooths the surface without shrinking it significantly ³³ ³⁴. Traditional Laplacian smoothing can shrink features, but Taubin's method avoids that by alternating smoothing passes ³³.

If you have Open3D, you can call `mesh.filter_smooth_taubin(number_of_iterations=...)` on your mesh data. For example:

```
# mesh is an open3d.geometry.TriangleMesh
mesh = mesh.filter_smooth_taubin(number_of_iterations=10)
mesh.compute_vertex_normals()
```

A few iterations (5-10) of Taubin can substantially soften the roughness without blurring out the overall shape too much. The result is a slightly more organic, rounded surface, which **looks better through glass** (sharp spikes can cause strange refractions or just be weaker points).

Be careful not to over-smooth (don't eliminate important details like the ridge of a nose). It's a balance – you want to remove the “stalactite” effect of pixel-by-pixel bumps.

3D. (Optional) Remeshing for Uniform Triangles

Depending on how the mesh was created, the triangles might be stretched in some areas (for example, if the original grid was not fine enough in one dimension). A process called **isotropic remeshing** can re-tessellate the surface so that all triangles are roughly equal-sized and well-shaped. This helps ensure consistent quality and also it's good for some 3D tools to avoid very long skinny triangles.

Tools: - **Open3D** doesn't have a full remesh, but it has subdivide. You could subdivide the mesh further then smooth again. - **MeshLab** has a filter for isotropic remeshing (under remeshing, simplification). You specify an edge length and it will re-mesh accordingly. - **Trimesh** or **PyMeshLab** in Python can perform remeshing. For instance, PyMeshLab (which uses the MeshLab core) has functions to remesh with a target edge length.

You can consider this step if the initial grid was too coarse or uneven. If your initial plane was already high-res (like vertices per pixel), you might skip further remeshing.

Either way, after displacement and smoothing, **recompute the vertex normals** (so that shading is correct if viewed in a 3D viewer – outward normals means the “front” of the relief is facing out of the crystal).

Now you have the final **mesh model**. It's essentially a 3D relief of your photo.

3E. Scale and Orientation, Then Export as STL/Obj

Make sure the mesh is at the correct real-world scale. We've been treating units as mm. The mesh width should be 80, height 60, depth ~20 (as per our example). STL doesn't embed units, but typically everyone assumes the numbers are in mm (since we intended that). Obj can have a scale but usually also taken as generic units.

Double-check orientation: In many 3D programs, the Z-axis is up. In our case, we want **+Z to represent “out of the front face of the crystal”** (toward the viewer when looking at the front of the block). Typically, when you give it to the laser software, you might align it so that the front face of the relief corresponds to the front face of the crystal. If Z is coming out of the image, that's fine. Some software might expect Y to be

depth – if so you may rotate the mesh accordingly. But generally, sticking to Z-up (height) is a good practice. We can always adjust orientation in Blender or the laser software later (for example, rotate so that the relief faces the correct direction).

Finally, export the mesh:

```
o3d.io.write_triangle_mesh("model.stl", mesh)
o3d.io.write_triangle_mesh("model.obj", mesh)
```

Now you have **model.stl** and/or **model.obj**. STL is widely accepted, OBJ as well – either one can be given to a vendor. Many laser engraving shops actually have software (like **Image3D** or **K9 conversion software**) that can take a 2.5D mesh and generate the point cloud (or .lss file) for the laser. If that's the case for your partner, you could even stop here and give them the mesh. But let's assume you want to generate the points yourself (which was the original plan).

Step 4: (Optional) 3D Face Refinement with Morphable Models

This step is optional and more advanced. If your business mainly does **human portraits**, one big quality boost can come from incorporating a **3D Morphable Model (3DMM)** for faces. What this means is using a statistical face model (like the Basel Face Model or FLAME) to fit a 3D face to your photo, which gives you a clean, anatomically plausible 3D shape of the person's face. You can then blend that with the depth-based relief to get more realistic features.

Why do this? Depth-from-image AI sometimes makes errors (e.g., a nose might be too flat, or eyes might get a weird depth). A 3DMM knows general human face geometry (eyes are spherical, noses protrude in a certain way, etc.). By fitting it, you essentially enforce a "ground truth" human shape. The downside is it requires an additional pipeline (detect landmarks, optimize the 3DMM to match them and the silhouette) which is complex.

If done, you would: - Use something like `face-alignment` to get 68 facial landmarks from the image. - Use a 3DMM fitting library (there are research implementations) to get a face mesh. - Adjust scale/position to match your relief. - Replace or blend the depth in the face region of your relief mesh with this model (perhaps blend the two height fields so that the face region is mostly the 3DMM shape, but the hair/clothing still come from AI depth). - The result: the face in the relief has correct curvature (eyes rounded, nose bridge well-defined, etc.), leading to a more life-like engraving.

If this sounds too much, you can skip it and maybe only consider it if you notice the AI depth giving unsatisfactory face shapes. Many commercial "2D-to-3D photo" companies do have proprietary methods like this to enhance faces.

Another related tweak: **Specular highlights adjustment**. Sometimes, purely geometric depth might not capture the way light hits the face in a photo. For engraving, it has been noted by some practitioners that slightly exaggerating certain features (like a nose or lips) can improve the perceived depth through the glass. One might take the depth map and locally boost it by a small percent at key features. This is a matter of experimentation and taste, so again optional.

Step 5: Generating the Laser Engraving Point Cloud

Now we have a nice mesh (the relief). Laser subsurface engraving doesn't use a mesh directly; it needs points inside the volume of the crystal where the laser will fire to create tiny fractures (dots). So we must convert our surface model into a set of points (**XYZ coordinates**) that represent the 3D "pixels" the laser will etch.

Point Sampling Strategy:

1. **On-surface sampling:** We want points on the surface of our relief, covering it evenly. If we only did this, the engraving would look like a thin shell of the face – which might actually be okay since from the front you'd see the face shape. But often, to give it some volume or sparkle, they add interior points.
2. **Volumetric or offset sampling:** We can create additional layers of points slightly behind the surface. For example, a common approach is to take the surface and push it a couple of millimeters "back" (deeper into the crystal), and sample points there too. This fills out some thickness, making the image visible from different viewing angles and adding depth.
3. **Uniform density and spacing:** As discussed, points should not be too close. Laser machines have a minimum spacing to avoid "clashes" (merged cracks) – typically on the order of 0.1 mm or 0.2 mm apart ¹⁶. We also don't want big gaps as that makes the engraving faint ³⁵. So a roughly uniform **point spacing** is desired, like a **blue-noise distribution** of points over the surface.

We will achieve this by **Poisson Disk sampling** on the mesh surface. Poisson disk sampling ensures that any two sample points are at least a certain distance apart, leading to an even distribution without clustering ¹⁸. Open3D provides a function for this: `sample_points_poisson_disk(mesh, number_of_points=..., init_factor=...)`. You can specify either an exact count or (better) a radius. Another approach: use a library or algorithm to do Poisson sampling until no more points can fit given a radius.

For example, using Open3D you might do:

```
pcd = mesh.sample_points_poisson_disk(number_of_points=500000, init_factor=5)
```

This tries to sample 500k points evenly on the surface ¹⁸. Instead of `number_of_points`, Open3D also allows specifying `init_factor` with an existing point cloud. It's a bit involved to guarantee a specific spacing, so sometimes it's easier to oversample and then filter by distance manually.

Alternatively, use **Trimesh** which has `sample.sample_surface_even(mesh, count, radius)` in its `trimesh.sample` module, which can ensure a radius (Poisson disc) by iterative elimination. There's also `point_cloud_utils` library that supports Poisson disk sampling by a radius ³⁶.

Spacing: Let's say we choose **0.1 mm as the minimum distance** between points. This value might come from the machine's spec (some can do finer, some need larger spacing). You can adjust accordingly. If too tight, risk of clashes; if too loose, image is faint ¹⁶.

So: - Sample the surface with radius = 0.1 mm (or spacing ~0.1). That gives a dense "shell" of points. - If desired, generate a second set: take the mesh and offset it inward by maybe 1 mm (like shrink it or move vertices along normals inward), and sample that too. Or an easier hack: take the surface points you just

sampled and for each point, add another point 0.5–1 mm behind it (in the +depth direction). This can simulate a slight thickness. However, be cautious to keep those also spaced out.

Combine point sets: Merge the surface points and any interior points. Now we have a collection of points that represent the 3D shape.

Apply margin (clipping): Ensure none of these points lie outside the safe volume of the crystal. If the crystal is 80×60×60 mm and we want a 3 mm clearance from all faces, then effectively we want points only within [3, 77] in X if width is 80, [3, 57] in Y if height is 60, and [3, 57] in Z if depth is 60 (assuming we position the relief starting at ~3mm from the front face). You might need to translate your model coordinates to the crystal's coordinates. For example, if your mesh's min Z was 0 at the front, you would add 3 mm so that the frontmost points are at Z=3 mm inside the block. Similarly, center or position it so leftmost X is at 3, etc. Then drop any points with coordinates outside [3, crystal_dim-3]. This ensures the laser doesn't hit the surface or edges of the crystal which could cause defects.

Jittering: A practical trick is to add a tiny random jitter (random offset) to each point's position, on the order of maybe ±0.01–0.02 mm. The reason is to avoid any subtle "layering" pattern. For instance, if your sampling happened in a perfectly grid-like way (unlikely with Poisson, but just in case), a small noise gives it a speckle distribution (blue noise). This can reduce the appearance of contours or machine path patterns in the final product.

In code, after you have an array of points:

```
points += np.random.normal(scale=0.01, size=points.shape)
```

This adds a Gaussian jitter with stddev = 0.01 mm (10 microns), which is tiny relative to the design (and well below laser focus size). You could also do uniform random in ±0.01.

At this stage, we have `points` as a NumPy array of shape (N,3) for N points.

5B. Exporting the Point Cloud

Finally, we output the point cloud to a file. Two common formats as discussed: - **XYZ file:** We can simply save each point as text line "X Y Z". Using numpy:

```
np.savetxt("points.xyz", points, fmt="%.3f")
```

This will save coordinates with 3 decimal places (micron-level detail is not needed typically). - **DXF file:** Using the `ezdxf` library, we can create a DXF and add points. For example:

```
import ezdxf
doc = ezdxf.new()
modelspace = doc.modelspace()
for (x,y,z) in points:
```

```
modelspace.add_point((x,y,z))
doc.saveas("points.dxf")
```

This will create a DXF where each point is a POINT entity at the given 3D coordinate.

Quality check on points: It's good to know how many points you ended up with. If it's, say, 300,000 points, that might be fine. If it's 5 million, that might be too many for the machine to handle efficiently (depending on machine and software). Some high-end SSLE machines can handle millions of points; others might require splitting into multiple layers or have longer processing times. Check with your manufacturer what a reasonable count is. Often, ~100k to 1M points is a typical range for a single crystal, but this varies. If needed, you can down-sample points a bit more (increase the spacing slightly or allow slightly larger spacing in low-detail areas).

According to one source, the accuracy of dot placement can be a few micrometers and dot size ~0.1 mm¹⁷, so our spacing of 0.1 mm is on that order. We also saw that point density can be used akin to grayscale – but in our method, we use a single point cloud to represent a surface, so we mostly rely on the geometry itself for the 3D effect rather than varying density for shading (except uniformly denser or sparser). The key is avoiding too high density that causes “clashes” (merged fractures)¹⁶.

Now you have **points.xyz** / **points.dxf** ready to go. This is what you'd give to the laser engraving company if they can convert it to their machine format (.lss or similar). Many engraving software allow importing DXF points directly into a laser program.

Step 6: Verification and Next Steps

Before sending the files out or engraving, it's wise to run through a checklist:

- **Verify the Mesh (STL/OBJ):** Open the `model.stl` or `model.obj` in a 3D viewer (MeshLab or Blender). Check that:
 - There are **no holes or missing faces** (the mesh should be a solid surface). Since it's a single displaced plane, it technically has open sides (the sides of the plane). That's okay for this purpose – we're not 3D printing it, and we only care about the front surface. But ensure the front surface itself is contiguous.
 - Normals are facing outward (in MeshLab, you can render with two-sided lighting off to see which side is the normal). If they're reversed (backwards), the engraving software might treat front as back. You can flip normals if needed.
 - The scale is correct – if you measure the mesh dimensions in MeshLab, are they 80×60×20 (ish) and not 8000×6000 etc. (common unit issues)? MeshLab has a ruler or you can see the axis grid.
- **Orientation:** Make sure up is up, etc., if that matters for how you place the piece in the machine or software.
- **Verify the Point Cloud:** You can also load `points.xyz` into MeshLab (it will just show a cloud of white dots). Look at it from the front – do you see the portrait or shape clearly in the distribution of points? Rotate it 3D – is the depth visible? Also check density by zooming in. There should be an even spread of points, not clusters or big gaps. If you see any strange outlier points floating off away from

the model, those should be removed (could be from background or noise – you can delete outliers in MeshLab by selecting them, or adjust your clipping logic in code).

- **Simulate the crystal volume:** It's helpful to imagine the 80×60×60 block around the points. You can create a transparent bounding box in Blender or MeshLab to see that points are indeed within bounds (3 mm from edges). In Blender, for example, you could create a cube 80×60×60 and then see that all points lie inside (Blender can import the XYZ if you use an addon or script, or import the OBJ of the mesh and ensure it's properly placed).
- **Small-scale test:** If possible, do a test engraving on a smaller block or a portion of the design to verify the settings (point spacing, etc.) produce a good result. This is more of a practical step once you have access to the machine, but it's a good idea as SSLE can sometimes show issues that are hard to predict (like internal reflections if too dense in one layer).
- **File format for delivery:** Since you might not have the .lss generator yet, coordinate with the engraver what they prefer. If they have an **internal software** that takes a 3D model, you might just send the STL/OBJ. They might generate the point cloud with their proprietary tool (because some laser systems come with software that converts STL into the appropriate laser points pattern automatically). On the other hand, if they expect you to provide the point cloud, a DXF or XYZ is what you have. Many professionals actually just send the 2.5D depth image or an STL to the service and the service does the rest. But doing it yourself gives you more control as we've done.

Finally, once everything checks out, you'll hand off the files to your partner or run it in the machine. **Make sure to specify to them the orientation** (which side is front) so the engraving isn't mirrored or flipped. Usually, you'd engrave from the back of the crystal so that when viewed from the front, the image is correct (because the laser focuses inside from the back). This might mean you need to mirror the model in depth. For example, many services ask for a mirror image because they invert it when engraving from the back. Check this detail: if engraving from the *back* face inward, you'll want the relief mirrored left-right (like a reversal) so that when viewed from front it looks normal. If engraving from the front face inward, you don't mirror. This is a nuance of how the laser machine is set up. It's worth clarifying with a small test.

At this point: Congratulations, you have created an SSLE-ready 3D model from a 2D photo! You went from a flat image to an **engraver-friendly point cloud** all with your own pipeline.

Using Blender and MeshLab in This Workflow

Now that you have the technical pipeline, a quick word on the GUI tools:

Blender – Blender is a powerful 3D modeling and animation suite. In our context, you can use Blender to: - **Visualize the mesh and point cloud:** Import your STL/OBJ to inspect it in detail (Blender can easily open OBJ/STL via File > Import). - **Apply the displacement within Blender (alternative method):** Blender has a **Displace Modifier** that can take a texture (your depth map image) and displace a subdivided plane. Steps in Blender would be: create plane, scale to aspect, subdivide (or add a Subdivision Surface modifier with levels to get high poly), add a Displace modifier, select "New Texture", in Texture properties open your depth.png (as an image texture), then adjust strength (this strength would correspond to the max height, e.g., 20 mm). This way, you can see and tweak the relief interactively. Blender 4.x might even have new geometry node

tools for this. There are also add-ons (like TrueDepth) that automate image-to-3D in Blender ³⁷. Using Blender might be more intuitive if you like visual feedback. - **Clean up and sculpting:** If the depth-based mesh has imperfections, you can enter Sculpt Mode in Blender and gently smooth or flatten areas as needed by hand. - **Retopology or Decimation:** Blender can decimate the mesh (reduce triangle count) if needed, or do remeshing (Voxel remesh or Quad remesh) if you want a different topology. - **Export formats:** Blender can export to STL, OBJ, PLY, etc. So you could do the entire pipeline in Blender nodes/modifiers and just export the final mesh.

MeshLab – MeshLab is a lighter-weight tool focused on processing meshes and point clouds: - **Viewing point clouds:** You can drag-and-drop the XYZ or DXF (sometimes needs DXF import plugin) into MeshLab to see the points. MeshLab can handle millions of points (depending on your RAM) and you can rotate around to inspect. - **Filters:** MeshLab has many built-in filters (some of which we did via Python): - Smoothing (Laplacian smooth, HC smooth, Taubin smooth) ³⁸ – you can apply these to the mesh and see results immediately. - Remeshing, simplification – e.g., “isotropic explicit remeshing” to even out triangle size, or “quadric edge collapse decimation” to reduce triangles while preserving shape. - Sampling – MeshLab can sample points on a mesh (there’s a filter “Sampling -> Poisson-disk Sampling” which will create a point cloud from the mesh with a specified number of points or radius). - Hole filling, normals flipping, scaling – useful utilities if your mesh had any issues. - **Conversion:** You can use MeshLab to convert between formats (say you had a PLY, you can save as STL, etc.). - **No Undo:** Caution, MeshLab doesn’t have undo on many operations, so work on copies of your mesh or save different stages.

Both tools complement the coding approach. For example, you might generate a depth map and initial mesh via code, then load it in Blender to fine-tune details, then sample points using MeshLab’s filter instead of writing your own, etc. As a programmer, you might prefer the full control of code (which is great for automation), but for intuition and one-off adjustments, the GUI can help you see what’s going on.

At this point, you have a comprehensive overview of the process. You’ve learned what STL/OBJ/XYZ/DXF files are and that **the quality of 3D output depends on the content (mesh resolution, point density) rather than the file format** ⁸. You set up a Python environment on Windows and used it to: - Enhance the image quality (Real-ESRGAN upscaling) ²² ²⁵. - Remove the background (rembg) ²⁷. - Compute a depth map using MiDaS (a high-accuracy model for single-image depth) ²⁸. - Build a 3D mesh by displacing a plane with that depth, taking care to scale to real units and smooth the geometry. - Sample a well-distributed point cloud from the mesh for laser engraving, leveraging Poisson-disk sampling to avoid point clustering ¹⁸. - Export results in formats the industry can use (STL/OBJ for surfaces, XYZ/DXF for points). - Verified the output using visualization tools, understanding the limitations of the laser process (dot size, spacing) ¹⁵.

All of this ensures your final laser-engraved crystal will have a clear, high-quality 3D image inside, with minimal guesswork or trial-and-error.

Next Steps / Further Learning: If you plan to do this regularly, you might consolidate these steps into a single Python script or even a simple application. You could make a CLI where you input a photo and get out an STL and XYZ. You might also explore training or using newer depth models as they come, or specialized ones for faces or specific objects for even better results. Keep an eye on the SSLE community for tips (as it’s a niche but passionate field).

Lastly, practice on a few images and perhaps have test engravings done to refine parameters like the depth scaling (Z_max) or point spacing to match your specific crystal size and laser machine's capabilities.

Should I provide a **detailed step-by-step guide on using Blender and MeshLab**, explaining their interfaces, relevant features, and how exactly to perform the operations mentioned (like displacement, smoothing, sampling) to help you further?

1 2 3 4 5 6 7 11 12 13 File Formats Explained for Makers: The Essential Guide to STL vs DXF vs OBJ - BeeGraphy Blog

<https://beegraphy.com/blog/file-formats-stl-dxf-obj/>

8 14 Has anyone seen any difference between an .STL file and an .OBJ in Prusa slicer? : r/prusa3d

https://www.reddit.com/r/prusa3d/comments/148teom/has_anyone_seen_any_difference_between_an_stl/

9 10 What is a Point Cloud?

<https://holocreators.com/blog/what-is-a-point-cloud/>

15 16 17 19 35 Sub-Surface 3D Laser Engraving: A Brief Explanation - FacFox Docs

<https://facfox.com/docs/kb/sub-surface-3d-laser-engraving-a-brief-explanation>

18 33 34 Mesh - Open3D 0.19.0 documentation

<https://www.open3d.org/docs/release/tutorial/geometry/mesh.html>

20 21 27 GitHub - danielgatis/rembg: Rembg is a tool to remove images background

<https://github.com/danielgatis/rembg>

22 23 24 py-real-esrgan · PyPI

<https://pypi.org/project/py-real-esrgan/>

25 26 Experimenting with Image to STL : r/lasercutting

https://www.reddit.com/r/lasercutting/comments/1b12lf5/experimenting_with_image_to_stl/

28 29 30 31 32 MiDaS – PyTorch

https://pytorch.org/hub/intelisl_midas_v2/

36 Resampling Point Clouds - Point Cloud Utils

https://www.fwilliams.info/point-cloud-utils/sections/point_cloud_resampling/

37 Turn 2D Images into 3D in a Few Clicks — True Depth Add-on for ...

<https://www.creativeshrimp.com/turn-2d-images-into-3d-in-a-few-clicks-true-depth-add-on-for-blender.html>

38 How to Cleanup and Refine Meshes in MeshLab - Utilities Online

<https://www.utilities-online.info/blogs/how-to-cleanup-and-refine-meshes-in-meshlab>